



**SIMATS**  
**ENGINEERING**



**SIMATS**  
Saveetha Institute of Medical And Technical Sciences  
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

**SMART HOSPITAL BED MANAGEMENT SYSTEM**

***A Runtime Polymorphism Based C++ Solution for Dynamic Patient Classification, Bed Allocation, Monitoring and Billing***

Course Name: **OBJECT ORIENTED PROGRAMMING (OOP) IN C++**

Course Code: **DSA0101**

to the award of the degree of

**BACHELOR OF ENGINEERING / BACHELOR OF TECHNOLOGY**

IN

**COMPUTER SCIENCE ENGINEERING**

**Submitted by**

J. Raksakanth (1911260099)

**Under the Supervision of**

Dr. Parimala Kanaga Devan K

Professor

**SIMATS ENGINEERING**

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

**SEPTEMBER 2026**

## **CERTIFICATE**

This is to certify that the project report entitled “**SMART HOSPITAL BED MANAGEMENT SYSTEM: A Runtime Polymorphism Based C++ Solution for Dynamic Patient Classification, Bed Allocation, Monitoring and Billing**” is a bona fide record of work carried out by **J.RAKSAKANTH**, Register Number **1911260099**, in fulfillment of the requirements of the course **COMPUTER SCIENCE AND ENGINEERING** during the academic year **2026-2029**.

The project demonstrates the application of Object-Oriented Programming principles using C++, with particular emphasis on Runtime Polymorphism, inheritance, abstraction, encapsulation, virtual functions, smart pointers, STL collections, and operator overloading.

The work presented in this report has been completed under the guidance and supervision of the undersigned.

### **COURSE COORDINATOR**

Dr. C. Sivasankar  
K  
Professor  
Department of CSE,  
SIMATS Engineering,  
Chennai – 602105.

### **COURSE FACULTY**

Dr. Parimala kanaga Devan  
  
Professor  
Department of CSE,  
SIMATS Engineering,  
Chennai – 602105

## **DECLARATION**

I, **J.RAKSAKANTH**, Register Number **1911260099**, Department of **COMPUTER SCIENCE ENGINEERING**, hereby declare that the project report entitled **“SMART HOSPITAL BED MANAGEMENT SYSTEM: A Runtime Polymorphism Based C++ Solution for Dynamic Patient Classification, Bed Allocation, Monitoring and Billing”** is an authentic record of the work carried out by me under the guidance of Dr. Parimala Kanaga Devan K .

I further declare that this project has been developed as an academic implementation of Object-Oriented Programming concepts in C++. The project particularly demonstrates Runtime Polymorphism through an abstract Patient base class and specialized derived classes representing General, ICU, and Emergency patients.

I confirm that the work presented in this report has not been submitted previously, either wholly or partly, for the award of any degree, diploma, or academic qualification at any other institution.

All external references, theoretical concepts, software documentation, and other sources used during the development of this project have been acknowledged appropriately.

Place: **CHENNAI** Date: **03/09/2026**

Signature of Student:

**J.RAKSAKANTH**

**1911260099**

## **ACKNOWLEDGEMENT**

I express my sincere gratitude to, my course instructor and project guide, for providing valuable guidance, technical suggestions, continuous encouragement, and constructive feedback throughout the development of the Smart Hospital Bed Management System.

The guidance received during the study of Object-Oriented Programming, particularly in the areas of inheritance, abstraction, virtual functions, dynamic binding, and runtime polymorphism, played an important role in designing the architecture of this project.

I would also like to thank Dr. C. Sivasankar Professor Department of CSE,, and the management SIMATS ENGINEERING for providing the required laboratory facilities, computer systems, software tools, and academic environment necessary for completing this project.

I extend my sincere appreciation to my classmates and friends who provided useful suggestions during the design, coding, testing, and debugging stages of the application.

Finally, I thank my family for their continuous encouragement and support throughout the completion of this academic project.

Student Name:

J.RAKSAKANTH

Register number:

1911260099

## ABSTRACT

Hospital bed management is an important administrative activity because hospitals must continuously coordinate available beds with patients having different levels of care requirements. General patients, ICU patients, and Emergency patients do not require the same type of bed, monitoring frequency, priority, or billing structure. A software system designed to manage these differences must therefore provide flexible mechanisms for representing patient-specific behavior.

The Smart Hospital Bed Management System is a console-based C++ application developed using Object-Oriented Programming principles. The central design concept of the project is Runtime Polymorphism. Instead of implementing patient-specific behavior through large if-else or switch structures, the system defines an abstract base class named Patient and derives specialized classes such as GeneralPatient, ICUPatient, and EmergencyPatient.

The abstract Patient class defines a common interface through virtual functions such as `getPatientType()`, `getRequiredBedType()`, `getDailyCharge()`, `getServiceCharge()`, `getMonitoringPlan()`, and `getPriorityScore()`. Each derived patient class overrides these functions according to its specific requirements. The Hospital class stores heterogeneous patient objects using `std::vector<std::unique_ptr<Patient>>`. When the hospital controller calls a virtual function through a base-class pointer, C++ dynamically selects the appropriate derived implementation at runtime.

The system provides facilities for patient registration, patient display, dynamic bed allocation, monitoring-unit updates, billing calculation, patient discharge, bed release, admission comparison, and hospital utilization reporting.

The project also demonstrates other important C++ concepts including inheritance, encapsulation, abstraction, virtual destructors, smart pointers, STL vectors, constructors, operator overloading, modular multi-file development, input validation, and structured testing.

The project demonstrates that Runtime Polymorphism can reduce dependency between the central hospital controller and specific patient categories. If a new patient category is introduced in the future, a new derived class can implement the existing Patient interface without requiring major changes to the central hospital management logic.

**Keywords:** C++, Object-Oriented Programming, Runtime Polymorphism, Virtual Functions, Dynamic Binding, Hospital Bed Management, Inheritance, Abstraction, Smart Pointers.

## List of Tables

| Table No.  | Table Title                                         | Page |
|------------|-----------------------------------------------------|------|
| Table 10.1 | Operational Parameters of Patient Categories        | 11   |
| Table 14.1 | Hardware and Software Requirements                  | 13   |
| Table 18.1 | Virtual Function Mapping                            | 19   |
| Table 18.2 | Runtime Dispatch Analysis                           | 19   |
| Table 20.1 | Requirement-to-Implementation Mapping               | 20   |
| Table 27.1 | System Test Case Matrix                             | 27   |
| Table 31.1 | Procedural Approach versus Polymorphic Architecture | 30   |
| Table 42.1 | Final Requirement Verification Audit                | 40   |

## List of Figures

| Figure No.  | Figure Title                                                     | Page |
|-------------|------------------------------------------------------------------|------|
| Figure 15.1 | UML Use Case Diagram of the Smart Hospital Bed Management System | 14   |
| Figure 16.1 | System Architecture Diagram                                      | 16   |
| Figure 17.1 | UML Class Diagram                                                | 17   |
| Figure 23.1 | Overall System Execution Flowchart                               | 24   |
| Figure 25.1 | Multi-file Project Directory Structure                           | 26   |
| Figure 26.1 | C++ Compilation and Execution in VS Code                         | 27   |

## Table of Contents

| Section    | Topic                                        | Page |
|------------|----------------------------------------------|------|
| <b>9.</b>  | <b>Introduction</b>                          | 10   |
| 9.1        | Background                                   | 10   |
| 9.2        | Motivation                                   | 10   |
| 9.3        | Importance of Runtime Polymorphism           | 10   |
| <b>10.</b> | <b>Real-World Background</b>                 | 11   |
| 10.1       | Hospital Bed Management                      | 11   |
| 10.2       | Patient Categories                           | 11   |
| 10.3       | Operational Parameters                       | 11   |
| 10.4       | Problems with Manual Management              | 11   |
| 10.5       | Software Engineering Perspective             | 11   |
| <b>11.</b> | <b>Problem Statement</b>                     | 12   |
| <b>12.</b> | <b>Objectives</b>                            | 12   |
| 12.1       | Primary Objective                            | 12   |
| 12.2       | Secondary Objectives                         | 12   |
| <b>13.</b> | <b>Scope of the Project</b>                  | 13   |
| 13.1       | In-Scope Features                            | 13   |
| 13.2       | Out-of-Scope Features                        | 13   |
| 13.3       | System Boundary                              | 13   |
| <b>14.</b> | <b>System Requirements</b>                   | 13   |
| 14.1       | Hardware Requirements                        | 13   |
| 14.2       | Software Requirements                        | 13   |
| 14.3       | Development Environment                      | 13   |
| 14.4       | C++ Standard                                 | 13   |
| <b>15.</b> | <b>Proposed Solution</b>                     | 13   |
| 15.1       | Patient Registration                         | 14   |
| 15.2       | Dynamic Bed Allocation                       | 14   |
| 15.3       | Dynamic Billing                              | 14   |
| 15.4       | Use Case Diagram                             | 14   |
| <b>16.</b> | <b>System Architecture</b>                   | 14   |
| 16.1       | User Interface Layer                         | 15   |
| 16.2       | Controller Layer                             | 15   |
| 16.3       | Patient Layer                                | 15   |
| 16.4       | Bed Layer                                    | 15   |
| 16.5       | Admission Layer                              | 15   |
| 16.6       | Runtime Execution                            | 15   |
| <b>17.</b> | <b>Class Design</b>                          | 16   |
| 17.1       | Patient Class                                | 16   |
| 17.2       | GeneralPatient Class                         | 16   |
| 17.3       | ICUPatient Class                             | 16   |
| 17.4       | EmergencyPatient Class                       | 17   |
| 17.5       | HospitalBed Class                            | 17   |
| 17.6       | Admission Class                              | 17   |
| 17.7       | Hospital Class                               | 17   |
| <b>18.</b> | <b>Runtime Polymorphism – Main Concept</b>   | 18   |
| 18.1       | Definition                                   | 18   |
| 18.2       | Runtime Polymorphism in This Project         | 18   |
| 18.3       | Virtual Functions                            | 18   |
| 18.4       | Dynamic Bed Allocation                       | 18   |
| 18.5       | Dynamic Billing                              | 18   |
| 18.6       | Dynamic Monitoring                           | 18   |
| 18.7       | Dynamic Priority                             | 18   |
| 18.8       | Virtual Table Concept                        | 18   |
| 18.9       | Virtual Function Mapping                     | 19   |
| 18.10      | Why Runtime Polymorphism Is the Main Concept | 19   |
| <b>19.</b> | <b>Supporting OOP Concepts</b>               | 19   |
| 19.1       | Classes and Objects                          | 19   |
| 19.2       | Encapsulation                                | 19   |
| 19.3       | Inheritance                                  | 19   |
| 19.4       | Abstraction                                  | 19   |
| 19.5       | Pure Virtual Functions                       | 19   |
| 19.6       | Virtual Destructor                           | 20   |
| 19.7       | Smart Pointers                               | 20   |
| 19.8       | STL Vector                                   | 20   |
| 19.9       | Operator Overloading                         | 20   |
| 19.10      | Constructors                                 | 20   |
| <b>20.</b> | <b>Requirement to Implementation Mapping</b> | 20   |
| <b>21.</b> | <b>Algorithms</b>                            | 21   |
| 21.1       | Algorithm for Patient Registration           | 21   |
| 21.2       | Algorithm for Dynamic Bed Allocation         | 21   |
| 21.3       | Algorithm for Monitoring Update              | 21   |
| 21.4       | Algorithm for Billing                        | 21   |
| 21.5       | Algorithm for Patient Discharge              | 22   |



| Section | Topic                                          | Page |
|---------|------------------------------------------------|------|
| 21.6    | Algorithm for Admission Comparison             | 22   |
| 21.7    | Algorithm for Utilization Report               | 22   |
| 22.     | <b>Pseudocode</b>                              | 22   |
| 22.1    | Patient Base Class                             | 22   |
| 22.2    | GeneralPatient                                 | 23   |
| 22.3    | ICUPatient                                     | 23   |
| 22.4    | EmergencyPatient                               | 23   |
| 22.5    | Dynamic Bed Allocation                         | 24   |
| 22.6    | Dynamic Billing                                | 24   |
| 23.     | <b>Flowchart</b>                               | 24   |
| 24.     | <b>Detailed Module Implementation</b>          | 24   |
| 24.1    | Patient Management Module                      | 24   |
| 24.2    | Bed Management Module                          | 25   |
| 24.3    | Admission Module                               | 25   |
| 24.4    | Runtime Polymorphism Module                    | 25   |
| 24.5    | Billing Module                                 | 25   |
| 24.6    | Discharge Module                               | 25   |
| 24.7    | Reporting Module                               | 25   |
| 25.     | <b>Source Code Implementation</b>              | 26   |
| 25.1    | Project Directory Structure                    | 26   |
| 25.2    | Patient.h                                      | 26   |
| 25.3    | GeneralPatient                                 | 27   |
| 25.4    | ICUPatient                                     | 27   |
| 25.5    | EmergencyPatient                               | 27   |
| 25.6    | HospitalBed                                    | 27   |
| 25.7    | Admission                                      | 27   |
| 25.8    | Hospital                                       | 27   |
| 26.     | <b>Program Compilation and Execution</b>       | 27   |
| 26.1    | Compilation                                    | 27   |
| 26.2    | Execution                                      | 27   |
| 26.3    | Compilation Verification                       | 27   |
| 27.     | <b>Testing and Validation</b>                  | 27   |
| 27.1    | Functional Testing                             | 28   |
| 27.2    | Boundary Testing                               | 28   |
| 27.3    | Polymorphism Testing                           | 28   |
| 27.4    | Billing Testing                                | 28   |
| 27.5    | Discharge Testing                              | 28   |
| 28.     | <b>Test Execution Evidence</b>                 | 28   |
| 29.     | <b>Sample Program Output</b>                   | 28   |
| 29.1    | Sample ICU Registration                        | 28   |
| 29.2    | Sample Dynamic Bed Allocation                  | 29   |
| 29.3    | Sample Dynamic Billing                         | 29   |
| 30.     | <b>Design Decisions and Justification</b>      | 29   |
| 30.1    | Runtime Polymorphism                           | 29   |
| 30.2    | Abstract Patient Class                         | 29   |
| 30.3    | Derived Classes                                | 29   |
| 30.4    | Smart Pointers                                 | 30   |
| 30.5    | STL Vector                                     | 30   |
| 30.6    | Hospital Controller                            | 30   |
| 30.7    | Separate Bed and Admission Classes             | 30   |
| 30.8    | Operator Overloading                           | 30   |
| 31.     | <b>Comparison with Alternative Approach</b>    | 30   |
| 31.1    | Procedural Example                             | 30   |
| 31.2    | Polymorphic Example                            | 30   |
| 32.     | <b>Advantages</b>                              | 31   |
| 33.     | <b>Limitations</b>                             | 31   |
| 34.     | <b>Future Enhancements</b>                     | 31   |
| 35.     | <b>Ethical and Professional Considerations</b> | 32   |
| 36.     | <b>SDG Relevance</b>                           | 32   |
| 36.1    | Relationship with Healthcare Efficiency        | 32   |
| 36.2    | Resource Utilization                           | 32   |
| 36.3    | Emergency Resource Management                  | 32   |
| 37.     | <b>Learning Outcomes</b>                       | 32   |
| 38.     | <b>Student Reflection</b>                      | 33   |
| 39.     | <b>Conclusion</b>                              | 34   |
| 40.     | <b>References</b>                              | 35   |
| 41.     | <b>Appendix</b>                                | 36   |
| 42.     | <b>Final Requirement Audit</b>                 | 40   |

## 9. Introduction

### 9.1 Background

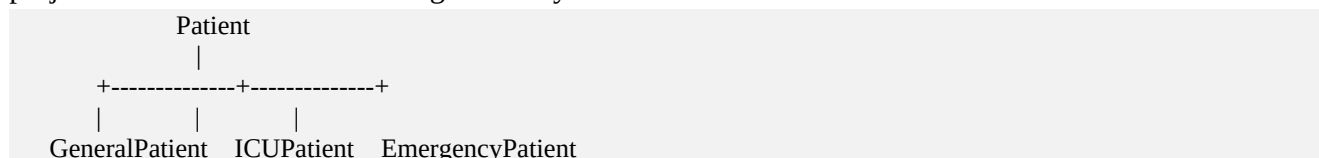
Hospital management involves the coordination of several resources including doctors, nurses, equipment, rooms, beds, emergency facilities, and patient records. Among these resources, hospital beds are particularly important because bed availability directly affects the ability of a hospital to admit and manage patients.

Different patients require different levels of care. A stable patient may require a standard general ward bed and periodic monitoring. A critically ill patient may require an ICU bed with continuous monitoring. An emergency patient may require rapid assessment and an emergency observation bed.

A software system designed for hospital bed management must therefore be able to recognize these differences and apply the appropriate operational rules.

From a programming perspective, this creates a problem involving heterogeneous objects with common characteristics but different behaviors.

Object-Oriented Programming provides a natural solution to this problem. A common abstract class can define the characteristics that all patients share, while derived classes can define specialized behavior. The project therefore uses the following hierarchy:



The most important design principle in this project is Runtime Polymorphism. Instead of asking the hospital controller:

```
Is this a General patient?
Is this an ICU patient?
Is this an Emergency patient?
```

and then executing separate conditional logic, the controller simply asks the patient object for the required behavior. For example:

```
patient->getRequiredBedType();
```

The actual function executed depends on the runtime type of the object. This approach creates a clean separation between the hospital management process and patient-specific behavior.

### 9.2 Motivation

The motivation for developing this project comes from the need to demonstrate how theoretical OOP concepts can be applied to a practical resource-management problem.

The project combines patient classification, bed allocation, monitoring management, billing, admission tracking, discharge, and resource utilization with inheritance, abstraction, encapsulation, runtime polymorphism, virtual functions, smart pointers, STL, and operator overloading.

Thus, the system acts as both a functional hospital-management simulation and a practical demonstration of advanced C++ OOP design.

### 9.3 Importance of Runtime Polymorphism

Runtime polymorphism is important because patient-specific behavior can change while the main hospital management logic remains the same. For example, the hospital controller does not need separate billing algorithms for every patient category. It can execute:

```
double dailyRate = patient->getDailyCharge();
double serviceRate = patient->getServiceCharge();
```

The correct values are obtained dynamically. This makes the software easier to maintain and extend.

## 10. Real-World Background

### 10.1 Hospital Bed Management

Hospital beds are limited physical resources. At any particular time, a hospital may have available beds, occupied beds, beds reserved for specialized care, or beds temporarily unavailable.

A bed-management system should therefore answer questions such as:

- Which beds are currently available?
- Which patient requires which type of bed?
- Which beds are occupied?
- Which patient is assigned to each bed?
- How long has the patient stayed?
- What monitoring services have been recorded?
- What is the estimated or final bill?

A centralized software system can simplify these tasks.

### 10.2 Patient Categories

The project considers three major patient categories.

**General Patient.** A General Patient represents a patient who requires standard hospital care. Typical characteristics include a general ward bed, lower priority, standard nurse checks, lower daily room charge, and basic service charges.

**ICU Patient.** An ICU Patient represents a critically ill patient requiring intensive care. Typical characteristics include an ICU bed, high priority, continuous telemetry, specialized nursing, higher daily room charge, and higher service charges.

**Emergency Patient.** An Emergency Patient represents a patient requiring immediate assessment and stabilization. Typical characteristics include an emergency bed, high priority, rapid triage, frequent assessment, and emergency-specific charges.

### 10.3 Operational Parameters

**Table 10.1: Operational Parameters of Patient Categories**

| Parameter      | General Patient      | ICU Patient          | Emergency Patient            |
|----------------|----------------------|----------------------|------------------------------|
| Required Bed   | General              | ICU                  | Emergency                    |
| Priority Range | 1–3                  | 8–10                 | 7–9                          |
| Daily Charge   | \$150                | \$1,200              | \$500                        |
| Service Charge | \$25/unit            | \$150/unit           | \$75/unit                    |
| Monitoring     | Standard Nurse Check | Continuous Telemetry | Continuous Triage Evaluation |

These values are simulation parameters used for the academic project.

### 10.4 Problems with Manual Management

Manual hospital bed management can lead to delayed bed allocation, incorrect patient-bed matching, difficulty maintaining current bed status, billing calculation errors, difficulty tracking monitoring units, duplicate or incomplete records, and poor utilization reporting. Automation can reduce these administrative difficulties.

### 10.5 Software Engineering Perspective

The problem also demonstrates an important software engineering issue. Suppose the system uses:

```
if(patientType == "ICU")
{
    // ICU logic
}
else if(patientType == "General")
{
    // General logic
}
```

```
else
{
    // Emergency logic
}
```

If the system grows, these conditions may appear repeatedly in bed allocation, billing, monitoring, reporting, and priority calculation. This creates duplicated and tightly coupled code. Runtime polymorphism provides a more maintainable alternative.

## 11. Problem Statement

Hospitals must manage multiple categories of patients while maintaining correct bed allocation, monitoring information, admission records, billing information, and discharge status.

The major software problem is that each patient category has different behavior. A General Patient requires a General bed, while an ICU Patient requires an ICU bed. Similarly, the daily charges and service charges vary according to patient category.

A procedural implementation could solve these differences using many conditional statements. However, such an approach becomes increasingly difficult to maintain as new patient categories are introduced.

The objective of this project is therefore to design and implement a C++ Smart Hospital Bed Management System in which Runtime Polymorphism is the central mechanism for handling patient-specific behavior.

The system must register patients, store different patient objects using a common interface, allocate appropriate beds dynamically, update monitoring information, calculate billing dynamically, discharge patients, release beds, compare admission records, generate utilization reports, validate invalid input, and demonstrate major OOP concepts.

The central architectural requirement is that patient-specific behavior should be resolved using virtual functions rather than repeated type-checking logic.

## 12. Objectives

### 12.1 Primary Objective

To develop a modular C++ Smart Hospital Bed Management System that demonstrates Runtime Polymorphism as the main architectural mechanism for dynamic patient classification, bed allocation, monitoring, and billing.

### 12.2 Secondary Objectives

**Objective 1 – Patient Registration.** Provide a structured mechanism for registering General, ICU, and Emergency patients.

**Objective 2 – Dynamic Classification.** Represent patient categories through derived classes of an abstract Patient base class.

**Objective 3 – Dynamic Bed Allocation.** Use the virtual function `getRequiredBedType()` to determine the appropriate bed category.

**Objective 4 – Dynamic Billing.** Use virtual functions to obtain patient-specific daily and service rates.

**Objective 5 – Monitoring Management.** Maintain monitoring units associated with active admissions.

**Objective 6 – Admission Management.** Create and maintain admission records linking patients with beds.

**Objective 7 – Discharge Management.** Calculate the final bill, mark the admission inactive, and release the assigned bed.

**Objective 8 – Safe Memory Management.** Use `std::unique_ptr<Patient>` for automatic memory ownership and cleanup.

**Objective 9 – Demonstrate OOP Concepts.** Apply abstraction, encapsulation, inheritance, runtime polymorphism, virtual functions, virtual destructors, and operator overloading.

**Objective 10 – Testing.** Validate the system through normal cases, edge cases, and invalid input cases.

### 13. Scope of the Project

#### 13.1 In-Scope Features

The project includes patient registration; General, ICU, and Emergency patient management; polymorphic patient storage; bed availability tracking; dynamic bed allocation; monitoring unit updates; dynamic billing; admission records; admission comparison; patient discharge; bed release; hospital utilization reporting; input validation; and console-based interaction.

#### 13.2 Out-of-Scope Features

The current academic implementation does not include database integration, cloud storage, a mobile application, a web application, real hospital network integration, real medical devices, real-time physiological sensors, electronic medical record integration, multi-user networking, authentication, production medical billing, or clinical decision-making. The application is an educational software simulation.

#### 13.3 System Boundary

The system starts when the user launches the application and initializes the hospital environment. The system ends when the user selects the Exit option. All runtime data is maintained in memory during execution.

### 14. System Requirements

#### 14.1 Hardware Requirements

**Table 14.1: Hardware and Software Requirements**

| Requirement | Minimum           | Recommended       |
|-------------|-------------------|-------------------|
| Processor   | Dual-core CPU     | Quad-core CPU     |
| RAM         | 2 GB              | 4 GB or higher    |
| Storage     | 50 MB             | 100 MB or higher  |
| Display     | Standard monitor  | Full HD monitor   |
| Keyboard    | Standard keyboard | Standard keyboard |
| Mouse       | Optional          | Recommended       |

#### 14.2 Software Requirements

| Software             | Requirement                                       |
|----------------------|---------------------------------------------------|
| Operating System     | Windows / Linux / macOS                           |
| Programming Language | C++                                               |
| C++ Standard         | C++17 or higher                                   |
| Compiler             | GCC / G++ / Clang / MSVC                          |
| IDE                  | Visual Studio Code / Code::Blocks / CLion         |
| Terminal             | Command Prompt / PowerShell / Integrated Terminal |

#### 14.3 Development Environment

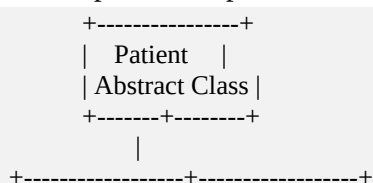
The project can be developed using Visual Studio Code with the C/C++ extension. The application uses multiple source and header files to maintain modularity.

#### 14.4 C++ Standard

C++17 is used because the project relies on modern C++ features including `std::unique_ptr` and `std::make_unique`.

### 15. Proposed Solution

The proposed solution uses an object-oriented architecture in which the Patient class defines a common interface and specialized patient classes provide individual implementations. The architecture is:





The hospital controller does not need to know the concrete patient class. It can simply use `Patient* patient;` and call `patient->getRequiredBedType();`, `patient->getDailyCharge();`, `patient->getServiceCharge();`, `patient->getMonitoringPlan();`, and `patient->getPriorityScore();`. The correct implementation is selected dynamically.

### 15.1 Patient Registration

When a user chooses a patient category, the system creates the corresponding derived object, for example `std::make_unique<GeneralPatient>(...)`, `std::make_unique<ICUPatient>(...)`, or `std::make_unique<EmergencyPatient>(...)`. The resulting smart pointer is stored in `std::vector<std::unique_ptr<Patient>> patients;`.

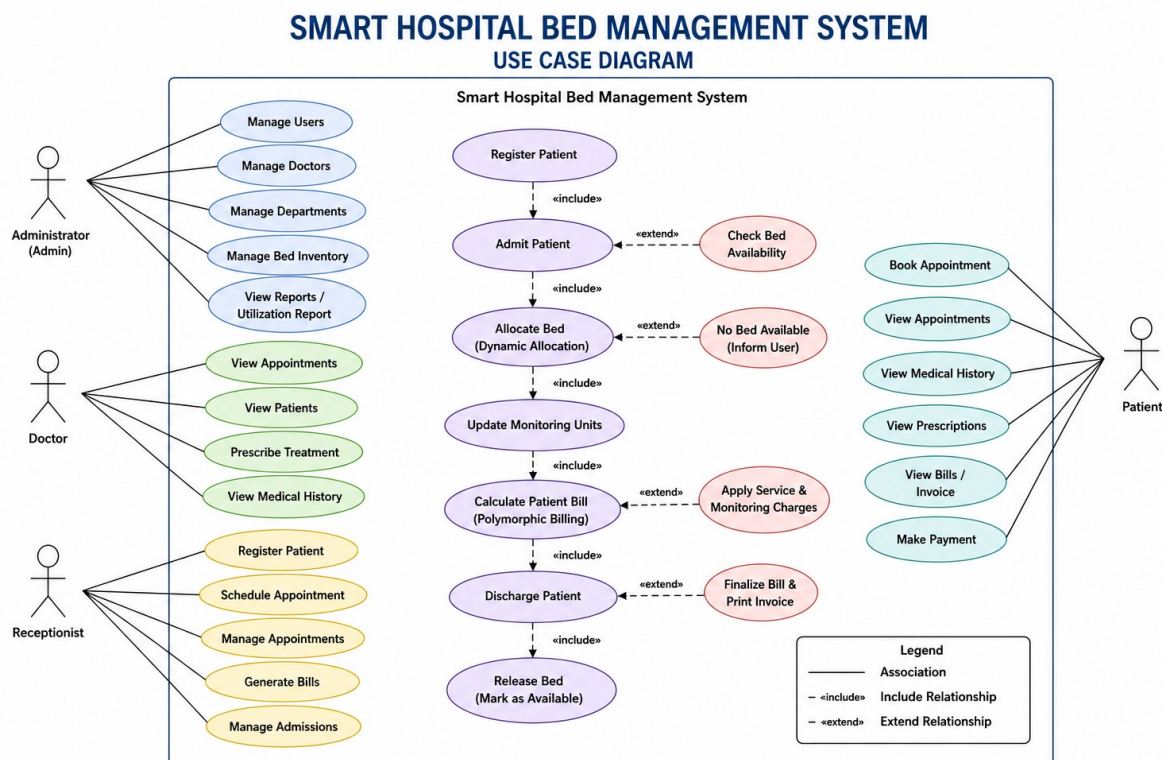
### 15.2 Dynamic Bed Allocation

During admission, the system executes `std::string requiredType = patient->getRequiredBedType();`. The result determines which type of bed should be searched.

### 15.3 Dynamic Billing

During billing, the system executes `double dailyRate = patient->getDailyCharge();` and `double serviceRate = patient->getServiceCharge();`. The correct rate is selected according to the runtime patient object.

### 15.4 Use Case Diagram



**Figure 15.1: UML Use Case Diagram of the Smart Hospital Bed Management System**

The major actor is the Hospital Staff / System User. Main use cases include Register Patient, View Patients, View Bed Status, Admit Patient, Update Monitoring, Calculate Bill, Discharge Patient, Compare Admissions, Generate Utilization Report, and Exit System.

## 16. System Architecture

The system follows a modular architecture.

## 16.1 User Interface Layer

The user interacts with the system through a console menu. The interface allows the user to select operations such as registering patients, displaying patients, viewing beds, admitting patients, updating monitoring, calculating bills, discharging patients, and viewing reports.

## 16.2 Controller Layer

The Hospital class acts as the central controller. It manages:

```
std::vector<std::unique_ptr<Patient>> patients;  
std::vector<HospitalBed> beds;  
std::vector<Admission> admissions;
```

The controller coordinates interactions between patient objects, beds, and admissions.

## 16.3 Patient Layer

```
Patient  
├── GeneralPatient  
├── ICUPatient  
└── EmergencyPatient
```

## 16.4 Bed Layer

The HospitalBed class represents a physical bed. It stores the Bed ID, ward type, occupancy state, and assigned patient ID.

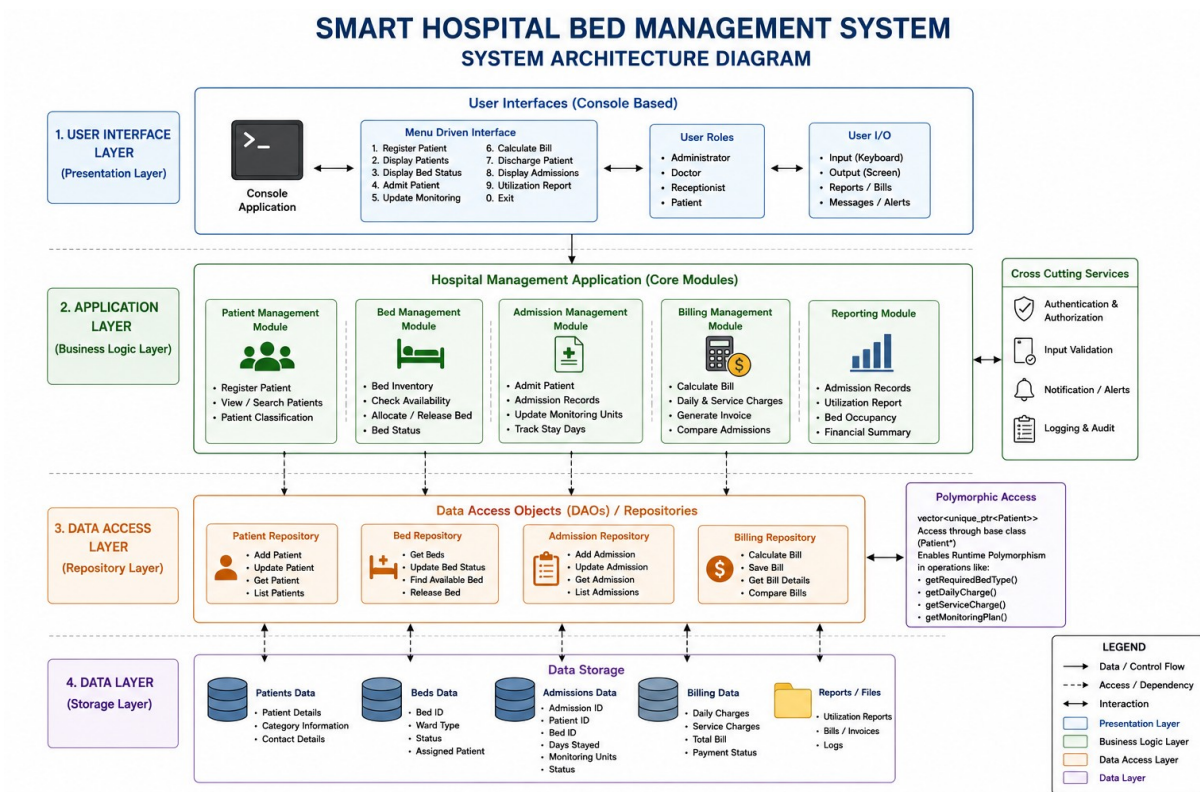
## 16.5 Admission Layer

The Admission class stores information about a patient's hospital stay, including admission ID, patient ID, bed ID, days stayed, monitoring units, active status, and total bill.

## 16.6 Runtime Execution

```
User  
↓  
Main Menu  
↓  
Hospital Controller  
↓  
Patient Base Pointer  
↓  
Virtual Function  
↓  
Derived Class Implementation  
↓  
Operation Result
```





**Figure 16.1: System Architecture of the Smart Hospital Bed Management System**

## 17. Class Design

### 17.1 Patient Class

Patient is the abstract base class. It contains common patient information: Patient ID, Name, Age, Gender, and Contact. It declares pure virtual functions:

```
virtual std::string getPatientType() const = 0;
virtual std::string getRequiredBedType() const = 0;
virtual double getDailyCharge() const = 0;
virtual double getServiceCharge() const = 0;
virtual std::string getMonitoringPlan() const = 0;
virtual int getPriorityScore() const = 0;
```

It also contains a virtual destructor:

```
virtual ~Patient() = default;
```

This is essential when derived objects are handled through base-class pointers.

### 17.2 GeneralPatient

GeneralPatient publicly inherits from Patient. Additional attributes may include insurance status and general ward preference. Its virtual functions provide General-specific behavior.

| Function             | General Result       |
|----------------------|----------------------|
| getPatientType()     | General Patient      |
| getRequiredBedType() | General              |
| getDailyCharge()     | \$150                |
| getServiceCharge()   | \$25                 |
| getMonitoringPlan()  | Standard Nurse Check |
| getPriorityScore()   | 1–3                  |

### 17.3 ICUPatient

ICUPatient publicly inherits from Patient. Additional attributes include ventilator requirement and criticality level. Its virtual functions provide ICU-specific behavior.

| Function             | ICU Result  |
|----------------------|-------------|
| getPatientType()     | ICU Patient |
| getRequiredBedType() | ICU         |
| getDailyCharge()     | \$1,200     |



| Function            | ICU Result           |
|---------------------|----------------------|
| getServiceCharge()  | \$150                |
| getMonitoringPlan() | Continuous Telemetry |
| getPriorityScore()  | 8–10                 |

## 17.4 EmergencyPatient

EmergencyPatient publicly inherits from Patient. Additional attributes include triage code and trauma level. Its virtual functions provide Emergency-specific behavior.

| Function             | Emergency Result             |
|----------------------|------------------------------|
| getPatientType()     | Emergency Patient            |
| getRequiredBedType() | Emergency                    |
| getDailyCharge()     | \$500                        |
| getServiceCharge()   | \$75                         |
| getMonitoringPlan()  | Continuous Triage Evaluation |
| getPriorityScore()   | 7–9                          |

## 17.5 HospitalBed

The HospitalBed class represents a physical hospital bed.

Attributes: bedId, wardType, isOccupied, assignedPatientId.

Operations: allocateBed(patientId), releaseBed(), isAvailable().

## 17.6 Admission

The Admission class represents a patient stay.

Attributes: admissionId, patientId, bedId, daysStayed, monitoringUnits, isActive, totalBill.

It also supports operator overloading. operator> can compare two admission records according to total billing. operator<< can provide formatted admission output.

## 17.7 Hospital

The Hospital class acts as the central management engine. Major operations include registerPatient(), admitPatient(), updateMonitoring(), calculateBill(), dischargePatient(), and generateUtilizationReport().

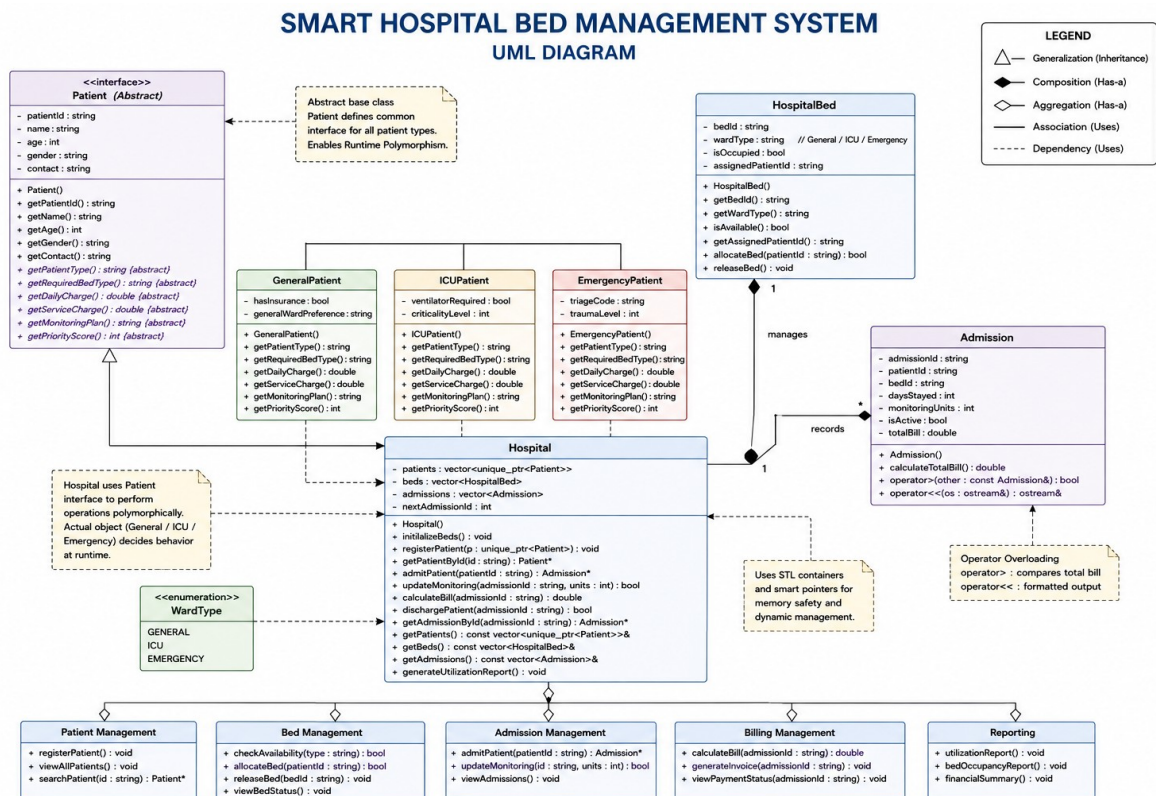


Figure 17.1: UML Class Diagram of the Smart Hospital Bed Management System

## 18. Runtime Polymorphism – Main Concept

### 18.1 Definition

Runtime Polymorphism is the ability of a program to determine which overridden function implementation should be executed at runtime. In C++, runtime polymorphism is commonly achieved through inheritance, virtual functions, base-class pointers or references, and dynamic binding.

### 18.2 Runtime Polymorphism in This Project

```
Patient*
|
+---- GeneralPatient
|
+---- ICUPatient
|
+---- EmergencyPatient
```

The same base-class pointer can refer to objects of different derived types. For example, Patient\* patient; could refer to GeneralPatient, ICUPatient, or EmergencyPatient. When the following statement is executed:

```
patient->getDailyCharge();
```

C++ determines which implementation should execute according to the runtime object.

### 18.3 Virtual Functions

The base class declares virtual double getDailyCharge() const = 0;. Each derived class overrides it: General returns 150, ICU returns 1200, Emergency returns 500. The hospital controller does not need three separate billing functions.

### 18.4 Dynamic Bed Allocation

The system executes std::string bedType = patient->getRequiredBedType();. If the patient is an ICU object, the runtime call returns “ICU”; if General, it returns “General”; if Emergency, it returns “Emergency”. The bed-management algorithm can then search for a matching available bed.

### 18.5 Dynamic Billing

Billing is calculated using: **Total Bill = (Days Stayed × Daily Charge) + (Monitoring Units × Service Charge).**

Suppose an ICU patient stays for 3 days and has 4 monitoring units:

```
Daily component = 3 × $1,200 = $3,600
Monitoring component = 4 × $150 = $600
Total = $4,200
```

The important point is that the billing function obtains the rates dynamically from the patient object.

### 18.6 Dynamic Monitoring

The monitoring plan is also determined polymorphically: General → Standard Nurse Check, ICU → Continuous Telemetry, Emergency → Continuous Triage Evaluation.

### 18.7 Dynamic Priority

The system can obtain a patient’s priority through patient->getPriorityScore();. The actual implementation depends on the runtime patient type.

### 18.8 Virtual Table Concept

A class containing virtual functions is conceptually associated with a virtual function table. A simplified representation is:

```
Patient*
|
v
Object
|
```

```

vptr
|
v
Virtual Table
|
+--> getPatientType()
+--> getRequiredBedType()
+--> getDailyCharge()
+--> getServiceCharge()
+--> getMonitoringPlan()
+--> getPriorityScore()

```

For an ICU object, the virtual dispatch mechanism selects the ICU implementations. This is a conceptual description of how the compiler and runtime implement dynamic dispatch, and it is not something manually programmed within the project.

## 18.9 Virtual Function Mapping

**Table 18.1: Virtual Function Mapping**

| Virtual Function     | GeneralPatient       | ICUPatient           | EmergencyPatient             |
|----------------------|----------------------|----------------------|------------------------------|
| getPatientType()     | General Patient      | ICU Patient          | Emergency Patient            |
| getRequiredBedType() | General              | ICU                  | Emergency                    |
| getDailyCharge()     | \$150                | \$1,200              | \$500                        |
| getServiceCharge()   | \$25                 | \$150                | \$75                         |
| getMonitoringPlan()  | Standard Nurse Check | Continuous Telemetry | Continuous Triage Evaluation |
| getPriorityScore()   | 1–3                  | 8–10                 | 7–9                          |

**Table 18.2: Runtime Dispatch Analysis**

| Call Site                     | Static Type | Runtime Type     | Function Selected                  |
|-------------------------------|-------------|------------------|------------------------------------|
| patient->getDailyCharge()     | Patient*    | GeneralPatient   | GeneralPatient::getDailyCharge()   |
| patient->getDailyCharge()     | Patient*    | ICUPatient       | ICUPatient::getDailyCharge()       |
| patient->getDailyCharge()     | Patient*    | EmergencyPatient | EmergencyPatient::getDailyCharge() |
| patient->getRequiredBedType() | Patient*    | ICUPatient       | ICUPatient::getRequiredBedType()   |

## 18.10 Why Runtime Polymorphism Is the Main Concept

Runtime polymorphism is not merely included as a theoretical feature. It directly controls patient classification, bed selection, billing, monitoring plan, priority, and administrative processing. Therefore, the main software architecture is based on dynamic behavior through the Patient interface.

## 19. Supporting OOP Concepts

### 19.1 Classes and Objects

The project models real-world entities using classes, for example Patient, HospitalBed, Admission, and Hospital. Objects are created from these class definitions during program execution.

### 19.2 Encapsulation

Encapsulation combines data and behavior inside classes while restricting direct access to internal data. For example, patient information can be protected using private: std::string id; std::string name; int age;. Public getter and setter methods can provide controlled access.

### 19.3 Inheritance

The specialized patient classes inherit common functionality from Patient, for example class ICUPatient : public Patient. This avoids unnecessary duplication of common patient information.

### 19.4 Abstraction

The abstract Patient class defines what operations every patient must provide without defining every implementation. This allows the hospital controller to work with the general concept of a patient.

### 19.5 Pure Virtual Functions

Functions declared with = 0 are pure virtual functions. They force derived classes to provide implementations.

## 19.6 Virtual Destructor

The base class uses virtual `~Patient() = default;`. This is important because derived objects are managed through base-class pointers.

## 19.7 Smart Pointers

The project uses `std::unique_ptr<Patient>`. This provides automatic memory ownership and cleanup.

## 19.8 STL Vector

The system uses `std::vector` to store dynamic collections, for example `std::vector<std::unique_ptr<Patient>>` and `std::vector<HospitalBed>`.

## 19.9 Operator Overloading

The Admission class uses operator overloading. `operator>` is used for comparing admission billing values. `operator<<` is used for formatted admission output.

## 19.10 Constructors

Constructors initialize objects when they are created. This ensures that patient, bed, and admission objects begin with meaningful values.

## 20. Requirement to Implementation Mapping

**Table 20.1: Requirement-to-Implementation Mapping**

| System Requirement        | OOP Concept          | Implementation                    |
|---------------------------|----------------------|-----------------------------------|
| Patient classification    | Runtime Polymorphism | Patient hierarchy                 |
| Common patient interface  | Abstraction          | Abstract Patient                  |
| Patient specialization    | Inheritance          | Derived classes                   |
| Patient-specific behavior | Virtual Functions    | Overridden methods                |
| Bed selection             | Runtime Polymorphism | <code>getRequiredBedType()</code> |
| Dynamic billing           | Runtime Polymorphism | Rate getters                      |
| Monitoring                | Runtime Polymorphism | <code>getMonitoringPlan()</code>  |
| Priority                  | Runtime Polymorphism | <code>getPriorityScore()</code>   |
| Memory management         | Smart Pointers       | <code>unique_ptr</code>           |
| Bed management            | Encapsulation        | <code>HospitalBed</code>          |
| Admission management      | Classes/Objects      | <code>Admission</code>            |
| Bill comparison           | Operator Overloading | <code>operator&gt;</code>         |
| Formatted output          | Operator Overloading | <code>operator&lt;&lt;</code>     |
| Dynamic collections       | STL                  | <code>vector</code>               |
| Safe destruction          | Virtual Destructor   | <code>~Patient()</code>           |

## **21. Algorithms**

### **21.1 Algorithm for Patient Registration**

**Input:** Patient details and category. **Output:** Registered patient object.

1. Start.
2. Request patient category.
3. Input patient ID.
4. Check whether the ID already exists.
5. If the ID exists, display an error.
6. Otherwise input name, age, gender, and contact.
7. If General is selected, input General-specific details.
8. Create GeneralPatient.
9. If ICU is selected, input ICU-specific details.
10. Create ICUPatient.
11. If Emergency is selected, input Emergency-specific details.
12. Create EmergencyPatient.
13. Store the object inside the polymorphic patient vector.
14. Display successful registration.
15. Stop.

### **21.2 Algorithm for Dynamic Bed Allocation**

1. Start.
2. Input patient ID.
3. Search for the patient.
4. If patient does not exist, display error.
5. Obtain the required bed type using patient->getRequiredBedType().
6. Search through all beds.
7. Check whether the bed is available.
8. Check whether the bed's ward type matches the required type.
9. If a matching bed is found, allocate it.
10. Create an admission record.
11. Store the admission.
12. Display successful allocation.
13. If no matching bed is available, display allocation failure.
14. Stop.

### **21.3 Algorithm for Monitoring Update**

1. Start.
2. Input admission ID.
3. Find the admission.
4. Validate that the admission is active.
5. Input monitoring units.
6. Add the new units to the admission.
7. Display the updated monitoring information.
8. Stop.

### **21.4 Algorithm for Billing**

1. Start.
2. Input admission ID.
3. Find the admission.
4. Find the associated patient.
5. Call getDailyCharge().

6. Call getServiceCharge().
7. Retrieve days stayed.
8. Retrieve monitoring units.
9. Calculate: Total Bill = (Days Stayed × Daily Charge) + (Monitoring Units × Service Charge).
10. Store the calculated amount.
11. Display the bill.
12. Stop.

### 21.5 Algorithm for Patient Discharge

1. Start.
2. Input admission ID.
3. Find active admission.
4. Find associated patient.
5. Calculate final bill dynamically.
6. Find assigned bed.
7. Release the bed.
8. Mark admission inactive.
9. Display final billing information.
10. Stop.

### 21.6 Algorithm for Admission Comparison

1. Start.
2. Select two admission records.
3. Compare total bills.
4. Invoke overloaded operator>.
5. Display which admission has the higher bill.
6. Stop.

### 21.7 Algorithm for Utilization Report

1. Start.
2. Count registered patients.
3. Count active admissions.
4. Count occupied beds.
5. Count available beds.
6. Group bed statistics by ward.
7. Calculate occupancy percentages.
8. Display summary.
9. Stop.

## 22. Pseudocode

### 22.1 Patient Base Class

```

CLASS Patient
    PROTECTED
        id
        name
        age
        gender
        contact

    PUBLIC
        getPatientType() = PURE VIRTUAL
        getRequiredBedType() = PURE VIRTUAL
        getDailyCharge() = PURE VIRTUAL
        getServiceCharge() = PURE VIRTUAL
        getMonitoringPlan() = PURE VIRTUAL

```

```
getPriorityScore() = PURE VIRTUAL
```

```
virtual destructor
```

```
END CLASS
```

## 22.2 GeneralPatient

```
CLASS GeneralPatient INHERITS Patient
```

```
ATTRIBUTES
```

```
hasInsurance
```

```
generalWardPreference
```

```
FUNCTIONS
```

```
getPatientType()
```

```
    RETURN "General Patient"
```

```
getRequiredBedType()
```

```
    RETURN "General"
```

```
getDailyCharge()
```

```
    RETURN 150
```

```
getServiceCharge()
```

```
    RETURN 25
```

```
getMonitoringPlan()
```

```
    RETURN "Standard Nurse Check"
```

```
getPriorityScore()
```

```
    RETURN value between 1 and 3
```

```
END CLASS
```

## 22.3 ICUPatient

```
CLASS ICUPatient INHERITS Patient
```

```
ATTRIBUTES
```

```
ventilatorRequired
```

```
criticalityLevel
```

```
FUNCTIONS
```

```
getPatientType()
```

```
    RETURN "ICU Patient"
```

```
getRequiredBedType()
```

```
    RETURN "ICU"
```

```
getDailyCharge()
```

```
    RETURN 1200
```

```
getServiceCharge()
```

```
    RETURN 150
```

```
getMonitoringPlan()
```

```
    RETURN "Continuous Telemetry"
```

```
getPriorityScore()
```

```
    RETURN value between 8 and 10
```

```
END CLASS
```

## 22.4 EmergencyPatient

```
CLASS EmergencyPatient INHERITS Patient
```

```
ATTRIBUTES
```

```
triageCode
```

```
traumaLevel
```

```
FUNCTIONS
```

```
getPatientType()
```

```
    RETURN "Emergency Patient"
```

```
getRequiredBedType()
```

```
    RETURN "Emergency"
```

```
getDailyCharge()
```

```
    RETURN 500
```

```
getServiceCharge()
```

```
    RETURN 75
```

```
getMonitoringPlan()
```

```
    RETURN "Continuous Triage Evaluation"
```

```
getPriorityScore()
```

```
        RETURN value between 7 and 9
END CLASS
```

## 22.5 Dynamic Bed Allocation

```
FUNCTION admitPatient(patientID)
    patient = findPatient(patientID)
    IF patient == NULL
        PRINT "Patient not found"
        RETURN
    END IF

    requiredBed = patient->getRequiredBedType()

    FOR each bed IN beds
        IF bed.isAvailable() AND bed.wardType == requiredBed
            bed.allocateBed(patientID)
            create Admission
            add Admission to admissions
            PRINT "Bed allocated successfully"
            RETURN
        END IF
    END FOR

    PRINT "No suitable bed available"
END FUNCTION
```

## 22.6 Dynamic Billing

```
FUNCTION calculateBill(admissionID)
    admission = findAdmission(admissionID)
    IF admission == NULL
        PRINT "Admission not found"
        RETURN
    END IF

    patient = findPatient(admission.patientID)
    dailyRate = patient->getDailyCharge()
    serviceRate = patient->getServiceCharge()

    total = admission.daysStayed * dailyRate
            + admission.monitoringUnits * serviceRate

    admission.totalBill = total
    PRINT total
END FUNCTION
```

## 23. Flowchart

The overall program execution is represented below.

### Figure 23.1: Overall System Execution Flowchart

## 24. Detailed Module Implementation

### 24.1 Patient Management Module

The Patient Management Module is responsible for registering and maintaining patient objects. The module creates derived patient objects according to the category selected by the user. The objects are stored polymorphically:

```
std::vector<std::unique_ptr<Patient>>> patients;
```

This allows the hospital controller to maintain General, ICU, and Emergency objects inside one collection.



## **24.2 Bed Management Module**

The Bed Management Module manages the physical hospital bed resources. Each HospitalBed object contains information about the bed identifier, ward type, occupancy state, and assigned patient. The module provides operations to allocate and release beds.

## **24.3 Admission Module**

The Admission Module creates records connecting Patient → Bed → Hospital Stay. The admission record maintains the admission ID, patient ID, bed ID, days stayed, monitoring units, bill, and active status.

## **24.4 Runtime Polymorphism Module**

This is the central module. The Hospital controller uses a base-class pointer, Patient\* patient;, and then invokes virtual functions. The implementation is selected at runtime.

## **24.5 Billing Module**

The Billing Module calculates costs using Room Charges + Service Charges. The rates are obtained dynamically from the patient's runtime class.

## **24.6 Discharge Module**

The Discharge Module locates the admission, calculates the final bill, releases the bed, marks the admission inactive, and displays the final details.

## **24.7 Reporting Module**

The Reporting Module summarizes total patients, active admissions, occupied beds, available beds, ward-level utilization, and patient category distribution.

## 25. Source Code Implementation

### 25.1 Project Directory Structure

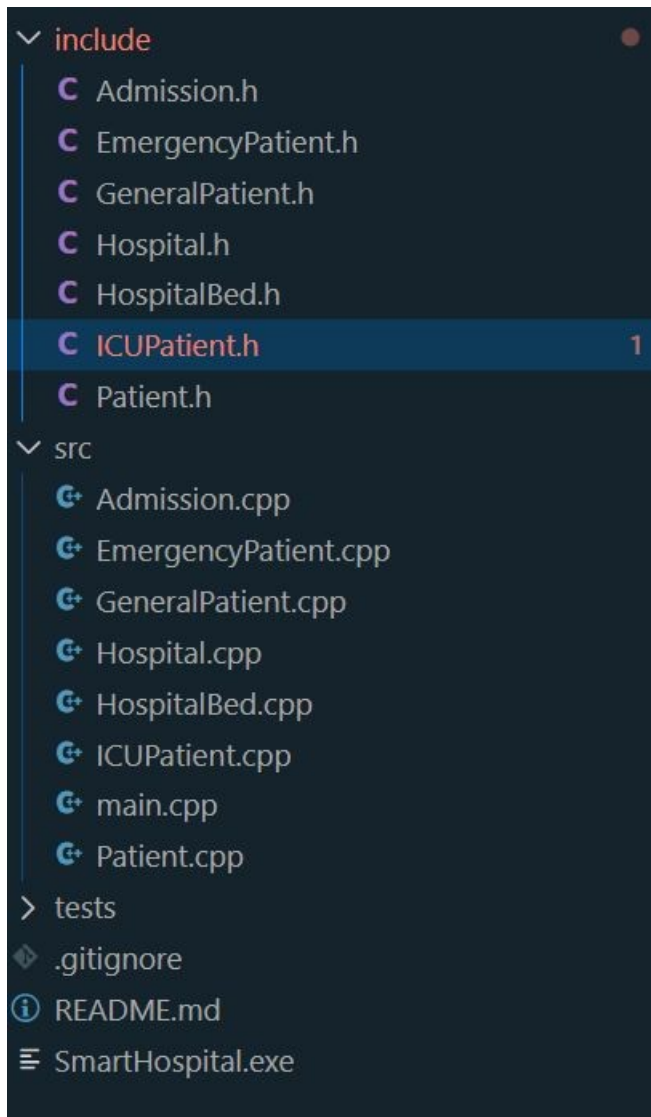


Figure 25.1: Multi-file project directory structure in VS Code

### 25.2 Patient.h

The abstract base class declares the common patient interface.

```
class Patient {  
protected:  
    std::string id;  
    std::string name;  
    int age;  
    std::string gender;  
    std::string contact;  
  
public:  
    virtual ~Patient() = default;  
  
    virtual std::string getPatientType() const = 0;  
    virtual std::string getRequiredBedType() const = 0;  
    virtual double getDailyCharge() const = 0;  
    virtual double getServiceCharge() const = 0;  
    virtual std::string getMonitoringPlan() const = 0;  
    virtual int getPriorityScore() const = 0;  
};
```

This file represents the foundation of Runtime Polymorphism.

### 25.3 GeneralPatient

The General Patient class overrides the virtual functions and provides General-specific behavior.

### 25.4 ICUPatient

The ICU Patient class provides specialized ICU requirements.

### 25.5 EmergencyPatient

The Emergency Patient class provides emergency-specific behavior.

### 25.6 HospitalBed

The bed class manages availability and assignment.

### 25.7 Admission

The admission class maintains stay information and supports overloaded operators.

### 25.8 Hospital

The Hospital class manages all collections and system operations. A key polymorphic operation is:

```
std::string neededBedType = patient->getRequiredBedType();
```

The Hospital class does not need to determine the concrete class manually.

## 26. Program Compilation and Execution

### 26.1 Compilation

The complete project can be compiled using:

```
g++ -std=c++17 -Iinclude src/main.cpp src/Patient.cpp src/GeneralPatient.cpp src/ICUPatient.cpp  
src/EmergencyPatient.cpp src/HospitalBed.cpp src/Admission.cpp src/Hospital.cpp -o SmartHospital.exe
```

The command compiles all source files and links them into the executable SmartHospital.exe.

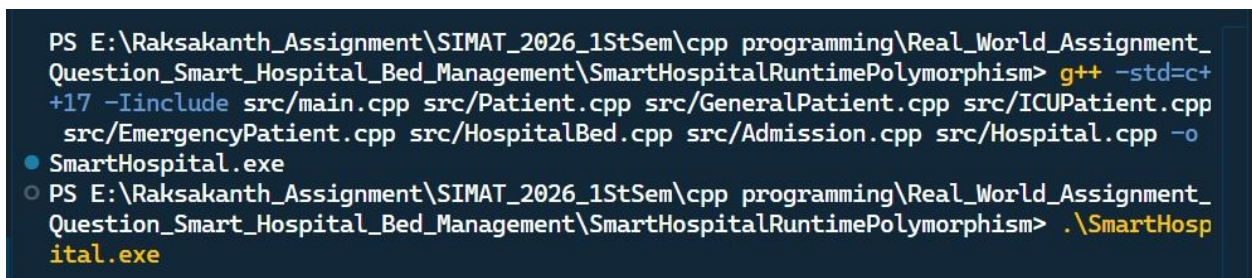
### 26.2 Execution

The program can be launched using:

```
.\SmartHospital.exe
```

### 26.3 Compilation Verification

The compilation should complete without compiler errors.



```
PS E:\Raksakanth_Assignment\SIMAT_2026_1StSem\cpp programming\Real_World_Assignment_
Question_Smart_Hospital_Bed_Management\SmartHospitalRuntimePolymorphism> g++ -std=c+
+17 -Iinclude src/main.cpp src/Patient.cpp src/GeneralPatient.cpp src/ICUPatient.cpp
src/EmergencyPatient.cpp src/HospitalBed.cpp src/Admission.cpp src/Hospital.cpp -o
SmartHospital.exe
PS E:\Raksakanth_Assignment\SIMAT_2026_1StSem\cpp programming\Real_World_Assignment_
Question_Smart_Hospital_Bed_Management\SmartHospitalRuntimePolymorphism> .\SmartHosp
ital.exe
```

Figure 26.1: Multi-file C++ compilation and execution in VS Code

## 27. Testing and Validation

Testing ensures that the system behaves according to its requirements. Testing is divided into functional testing, input validation, boundary testing, polymorphism testing, bed allocation testing, billing testing, discharge testing, and reporting testing.

Table 27.1: System Test Case Matrix

| Test ID | Feature                | Test Input              | Expected Result         |
|---------|------------------------|-------------------------|-------------------------|
| TC01    | General registration   | Valid General patient   | Registration successful |
| TC02    | ICU registration       | Valid ICU patient       | Registration successful |
| TC03    | Emergency registration | Valid Emergency patient | Registration successful |

| Test ID | Feature              | Test Input                | Expected Result        |
|---------|----------------------|---------------------------|------------------------|
| TC04    | Duplicate ID         | Existing patient ID       | Registration rejected  |
| TC05    | Invalid age          | Negative/out-of-range age | Input rejected         |
| TC06    | General allocation   | General patient           | General bed assigned   |
| TC07    | ICU allocation       | ICU patient               | ICU bed assigned       |
| TC08    | Emergency allocation | Emergency patient         | Emergency bed assigned |
| TC09    | Capacity exceeded    | No suitable bed           | Allocation rejected    |
| TC10    | Monitoring update    | Valid admission           | Monitoring updated     |
| TC11    | Dynamic billing      | ICU admission             | Correct ICU rate used  |
| TC12    | Discharge            | Active admission          | Patient discharged     |
| TC13    | Bed release          | Discharged patient        | Bed becomes available  |
| TC14    | Operator >           | Two admissions            | Correct comparison     |
| TC15    | Utilization report   | Report option             | Statistics displayed   |
| TC16    | Invalid menu         | Invalid option            | Error handled          |

## 27.1 Functional Testing

Functional testing verifies that each menu operation performs the intended task.

## 27.2 Boundary Testing

Boundary testing is applied to values such as age, priority, monitoring units, and menu choices.

## 27.3 Polymorphism Testing

Polymorphism is verified by registering different patient types and observing that the same base-class interface produces different results. For example, `patient->getRequiredBedType()`; must produce “General” for a General patient, “ICU” for an ICU patient, and “Emergency” for an Emergency patient.

## 27.4 Billing Testing

Billing is tested using  $\text{Total} = (\text{Days} \times \text{Daily Rate}) + (\text{Monitoring Units} \times \text{Service Rate})$ . Example (ICU):  $3 \text{ days} \times \$1,200 = \$3,600$ ;  $4 \text{ units} \times \$150 = \$600$ ; Total = \$4,200.

## 27.5 Discharge Testing

Discharge testing verifies that the bill is calculated, the admission becomes inactive, and the assigned bed becomes available.

## 29. Sample Program Output

The following is a representative example of the type of output expected from the application.

```
=====
SMART HOSPITAL BED MANAGEMENT SYSTEM
=====

1. Register New Patient
2. Display All Patients
3. Display Bed Status
4. Admit Patient
5. Update Monitoring Units
6. Calculate Patient Bill
7. Discharge Patient
8. Display Admission Records
9. Generate Hospital Utilization Report
0. Exit
```

Enter Choice:

### 29.1 Sample ICU Registration

Select Patient Category:

1. General
2. ICU
3. Emergency

Enter Choice: 2

Enter Patient ID: P102

Enter Name: Sarah

Enter Age: 52

Enter Gender: Female  
Enter Contact: 9876543210  
Ventilator Required: Yes  
Criticality Level: 4

SUCCESS: ICU Patient registered successfully.

## 29.2 Sample Dynamic Bed Allocation

Enter Patient ID for Admission: P102

[POLYMORPHIC DISPATCH]  
Calling getRequiredBedType()...

Required Bed Type: ICU

Searching available beds...

Available Bed Found: BED-ICU-01

SUCCESS:  
Patient Sarah allocated to BED-ICU-01

## 29.3 Sample Dynamic Billing

Enter Admission ID: ADM-101

[POLYMORPHIC DISPATCH]

Patient Type: ICU Patient  
Daily Rate: \$1200.00  
Service Rate: \$150.00

Days Stayed: 3  
Monitoring Units: 4

Room Charges:  
3 x \$1200.00 = \$3600.00

Monitoring Charges:  
4 x \$150.00 = \$600.00

-----  
TOTAL BILL: \$4200.00  
-----

The output demonstrates that the correct ICU rates are obtained through the patient interface rather than through a separate ICU-specific billing algorithm.

## 30. Design Decisions and Justification

### 30.1 Runtime Polymorphism

Runtime polymorphism was selected as the central design mechanism because different patient categories have different behavior. The controller can therefore use `Patient*` instead of depending on concrete classes.

### 30.2 Abstract Patient Class

Patient acts as a contract. Every derived patient class must provide implementations for the required virtual functions.

### 30.3 Derived Classes

The three derived classes encapsulate specialized behavior. This keeps patient-specific rules in the appropriate class.

### 30.4 Smart Pointers

`std::unique_ptr` provides automatic ownership and cleanup. This reduces the risk of memory leaks, double deletion, and dangling pointers.

### 30.5 STL Vector

Vectors provide convenient dynamic collections. The polymorphic patient collection is `std::vector<std::unique_ptr<Patient>>`.

### 30.6 Hospital Controller

The Hospital class coordinates operations while avoiding unnecessary knowledge of concrete patient types. This improves separation of responsibilities.

### 30.7 Separate Bed and Admission Classes

A hospital bed is a physical resource, while an admission represents a patient's stay. Separating these concepts improves the structure of the program.

### 30.8 Operator Overloading

Operator overloading makes admission comparison and output easier to read.

## 31. Comparison with Alternative Approach

**Table 31.1: Procedural Approach versus Runtime Polymorphism**

| Factor                        | Procedural Conditional Design | Polymorphic Design   |
|-------------------------------|-------------------------------|----------------------|
| Patient handling              | Type-based conditions         | Base-class interface |
| Bed allocation                | Conditional branches          | Virtual function     |
| Billing                       | Type checks                   | Virtual rate methods |
| Monitoring                    | Conditional logic             | Overridden method    |
| Extensibility                 | Requires modifying controller | Add derived class    |
| Maintainability               | Lower                         | Higher               |
| Code duplication              | Higher                        | Lower                |
| Separation of concerns        | Limited                       | Stronger             |
| Memory management             | May require manual handling   | Smart pointers       |
| Central controller dependency | High                          | Lower                |

### 31.1 Procedural Example

A procedural approach may use:

```
if(type == "General")
{
    rate = 150;
}
else if(type == "ICU")
{
    rate = 1200;
}
else
{
    rate = 500;
}
```

If another patient type is introduced, this code must be modified.

### 31.2 Polymorphic Example

The implemented approach uses:

```
double rate = patient->getDailyCharge();
```

The object determines its own rate. This demonstrates the practical benefit of runtime polymorphism.

## 32. Advantages

**32.1 Extensibility.** New patient categories can be introduced by creating new derived classes.

**32.2 Maintainability.** Patient-specific logic remains within the relevant class.

**32.3 Reduced Conditional Logic.** The controller does not require repeated type checks.

**32.4 Reusability.** The common Patient interface can be reused by many hospital operations.

**32.5 Memory Safety.** Smart pointers automatically manage dynamic memory.

**32.6 Modularity.** The multi-file structure separates interfaces and implementations.

**32.7 Clear Architecture.** The system has distinct responsibilities for patients, beds, admissions, hospital control, billing, and reporting.

**32.8 Demonstration of OOP.** The project demonstrates several important C++ concepts within one practical application.

## 33. Limitations

The current implementation has several limitations.

**33.1 No Database.** All records are maintained in memory. When the application terminates, runtime data is lost.

**33.2 Console Interface.** The application uses a command-line interface instead of a graphical interface.

**33.3 Single User.** The current application is designed for local single-user execution.

**33.4 No Real-Time Medical Devices.** Monitoring values are entered or simulated by the user. The project does not directly connect to medical sensors.

**33.5 Simplified Billing.** The billing model is intentionally simplified for academic demonstration. Real hospitals may have significantly more complex billing structures.

**33.6 Educational Scope.** The application is intended to demonstrate software engineering and C++ OOP concepts. It should not be considered a production clinical management system.

## 34. Future Enhancements

**34.1 Database Integration.** A database such as SQLite, MySQL, or PostgreSQL could be introduced to store patient and admission records permanently.

**34.2 Graphical User Interface.** A GUI could make bed availability and patient information easier to visualize.

**34.3 Web-Based Dashboard.** A web dashboard could allow authorized hospital staff to access bed information remotely.

**34.4 Authentication.** The system could implement secure user authentication. Different users could have different permissions.

**34.5 Role-Based Access.** Roles could include Administrator, Doctor, Nurse, Billing Officer, and Receptionist.

**34.6 Real-Time Bed Monitoring.** Sensors could provide automatic information about bed occupancy.

**34.7 Additional Patient Classes.** Future derived classes could include PediatricPatient, MaternityPatient, SurgicalPatient, and CardiacPatient. This would further demonstrate the extensibility of the polymorphic architecture.

**34.8 Hospital Networking.** Multiple departments could share bed availability information.

**34.9 Cloud Deployment.** The system could eventually become a cloud-connected hospital resource management application.

**34.10 Advanced Analytics.** Future versions could analyze bed utilization, average stay, ward demand, admission trends, and resource requirements.

### **35. Ethical and Professional Considerations**

Healthcare-related software requires responsible design because patient information can be sensitive and operational errors can have serious consequences.

**35.1 Patient Privacy.** Real-world implementations must protect personal and medical information.

**35.2 Data Security.** Sensitive information should be protected against unauthorized access.

**35.3 Data Integrity.** Patient and admission records must remain accurate.

**35.4 Fair Resource Allocation.** Automated bed allocation should be based on defined operational rules rather than unfair or arbitrary criteria.

**35.5 Reliable Billing.** Incorrect billing can cause administrative and financial problems. Therefore, billing calculations must be tested carefully.

**35.6 Input Validation.** Invalid patient information and invalid menu selections must be handled safely.

**35.7 Professional Responsibility.** Real healthcare applications require extensive validation, security testing, auditing, documentation, and compliance procedures. The current project is an academic simulation and does not claim clinical certification.

### **36. SDG Relevance**

#### **Sustainable Development Goal 3 – Good Health and Well-Being**

The project is related to United Nations Sustainable Development Goal 3: Good Health and Well-Being. Efficient healthcare resource management can support hospitals in using their available resources more effectively. The Smart Hospital Bed Management System demonstrates how software can organize bed availability, patient classification, patient admission, monitoring records, billing, and resource utilization.

Although the current implementation is an educational simulation, the underlying software engineering principles are relevant to healthcare resource-management systems.

#### **36.1 Relationship with Healthcare Efficiency**

A better-organized bed management process can help hospital administrators understand resource availability and allocation.

#### **36.2 Resource Utilization**

The utilization-reporting module demonstrates how software can summarize the current use of hospital beds.

#### **36.3 Emergency Resource Management**

The Emergency Patient category demonstrates the importance of distinguishing urgent resource requirements from standard admissions.

### **37. Learning Outcomes**

The development of this project provided practical understanding of several C++ and software-engineering concepts.

**37.1 Runtime Polymorphism.** The project demonstrated how virtual functions and base-class pointers can provide dynamic behavior.

**37.2 Abstract Classes.** The Patient class demonstrates how an abstract interface can define a common contract.

**37.3 Inheritance.** Derived classes reuse common patient attributes while implementing specialized behavior.



**37.4 Encapsulation.** Class data is protected through controlled access.

**37.5 Smart Pointers.** The project provided practical experience with `std::unique_ptr`.

**37.6 STL.** The project uses `std::vector` to manage dynamic collections.

**37.7 Operator Overloading.** The Admission class demonstrates custom operators.

**37.8 Multi-File Compilation.** The project demonstrates how .h and .cpp files are organized and compiled together.

**37.9 Algorithm Design.** The project required algorithms for registration, allocation, billing, monitoring, discharge, and reporting.

**37.10 Testing.** The test matrix helped identify normal, boundary, and invalid-input scenarios.

## **38. Student Reflection**

Developing the Smart Hospital Bed Management System provided practical experience in applying Object-Oriented Programming concepts to a real-world-inspired software problem.

Initially, the different patient categories could be viewed as separate cases requiring separate conditional statements. However, designing the project around Runtime Polymorphism provided a better way to represent the differences.

The most important learning outcome was understanding how an abstract base class can define a common interface while derived classes provide specialized behavior. For example, the Hospital class does not need to determine whether a patient is an ICU patient before requesting the required bed type. It simply calls `patient->getRequiredBedType()`. The runtime object provides the correct implementation. This helped demonstrate the practical meaning of dynamic binding.

The project also improved my understanding of memory management in modern C++. Using `std::unique_ptr<Patient>` made it possible to store heterogeneous derived objects while allowing automatic memory cleanup.

Another important learning experience was organizing the project into separate header and source files. Separating declarations from implementations improved the structure of the project and made debugging easier.

Testing the application also demonstrated that software design is not complete after writing the code. Each module must be validated using normal and invalid inputs.

Overall, this project helped connect theoretical concepts such as abstraction, inheritance, virtual functions, and polymorphism with a practical application involving hospital resource management.

### 39. Conclusion

The Smart Hospital Bed Management System demonstrates how Object-Oriented Programming and Runtime Polymorphism can be applied to a practical resource-management problem.

The system uses an abstract Patient class as the common interface for multiple patient categories. GeneralPatient, ICUPatient, and EmergencyPatient inherit from this base class and provide specialized implementations for bed requirements, billing rates, monitoring plans, patient classification, and priority.

The central Hospital controller works with patients through base-class pointers and smart pointers rather than depending directly on concrete patient classes. This allows virtual functions to determine the correct behavior dynamically at runtime.

Runtime polymorphism is therefore used directly in important system operations including patient classification, dynamic bed allocation, monitoring-plan selection, priority determination, and billing calculation.

The project also incorporates encapsulation, inheritance, abstraction, virtual destructors, smart pointers, STL vectors, constructors, operator overloading, input validation, and modular source organization.

The system demonstrates that a polymorphic architecture can reduce repeated conditional logic and make the software easier to extend. A future patient category can be implemented as a new derived class while preserving the common interface used by the Hospital controller.

Although the current application is a console-based academic simulation without persistent storage or real medical-device integration, it successfully demonstrates the intended C++ concepts.

Therefore, the project achieves its primary objective of demonstrating Runtime Polymorphism as the central architectural mechanism for dynamic hospital bed management.

## 40. References

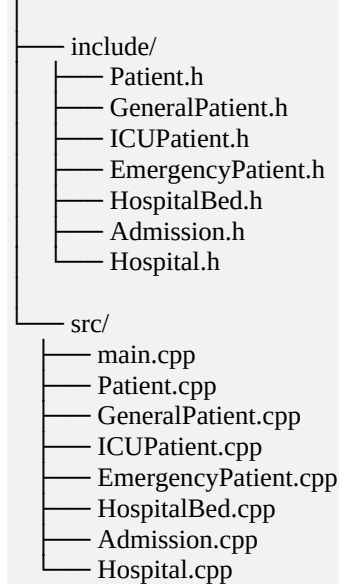
1. Stroustrup, B. *The C++ Programming Language*. 4th Edition, Addison-Wesley Professional.
2. Lippman, S. B., Lajoie, J., and Moo, B. E. *C++ Primer*. 5th Edition, Addison-Wesley Professional.
3. Meyers, S. *Effective Modern C++*. O'Reilly Media.
4. Gamma, E., Helm, R., Johnson, R., and Vlissides, J. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
5. ISO/IEC. *Programming Languages — C++*. International Standard.
6. cppreference. *Virtual Functions and Dynamic Binding in C++*.
7. cppreference. *std::unique\_ptr – C++ Standard Library Reference*.
8. cppreference. *std::vector – C++ Standard Library Reference*.
9. United Nations. *Sustainable Development Goal 3: Good Health and Well-Being*.

## 41. Appendix

### Appendix A – Complete Source Code Directory

The complete project source code is organized into the following directories:

SmartHospitalRuntimePolymorphism/



### Appendix B – Source Code Evidence

**Exhibit B.1 – Abstract Patient Interface.** [INSERT SCREENSHOT OF Patient.h HERE]. The screenshot should clearly show:

```
virtual std::string getRequiredBedType() const = 0;
virtual double getDailyCharge() const = 0;
virtual double getServiceCharge() const = 0;
```

*Caption: Exhibit B.1: Abstract Patient interface and pure virtual functions*

**Exhibit B.2 – Runtime Polymorphism in Hospital.cpp.** [INSERT SCREENSHOT OF Hospital.cpp HERE]. Highlight:

```
patient->getRequiredBedType();
patient->getDailyCharge();
patient->getServiceCharge();
```

*Caption: Exhibit B.2: Runtime polymorphic calls inside the Hospital controller*

### Appendix C – Screenshot Gallery

Insert the complete collection of actual execution screenshots here. Recommended screenshots: project folder structure; compilation; main menu; General registration; ICU registration; Emergency registration; patient display; bed status; General allocation; ICU allocation; Emergency allocation; monitoring update; billing; admission records; admission comparison; utilization report; discharge; invalid input.

### Appendix D – GitHub Repository

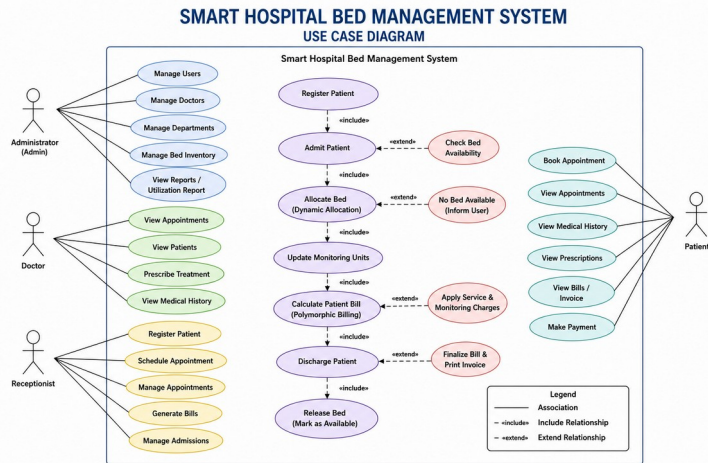
Project Repository: [INSERT GITHUB REPOSITORY URL HERE]

Repository Name: SmartHospitalRuntimePolymorphism

Description: C++17 Smart Hospital Bed Management System demonstrating Runtime Polymorphism, inheritance, abstraction, dynamic bed allocation, monitoring, billing, admission management, and hospital utilization reporting.

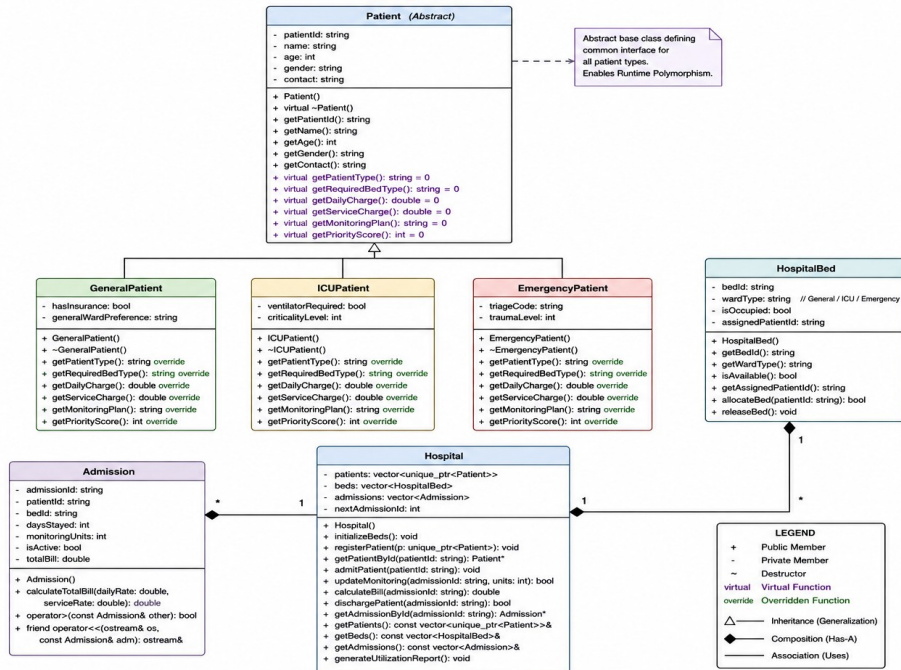
[INSERT GITHUB QR CODE HERE]

## Appendix E – Additional Diagrams

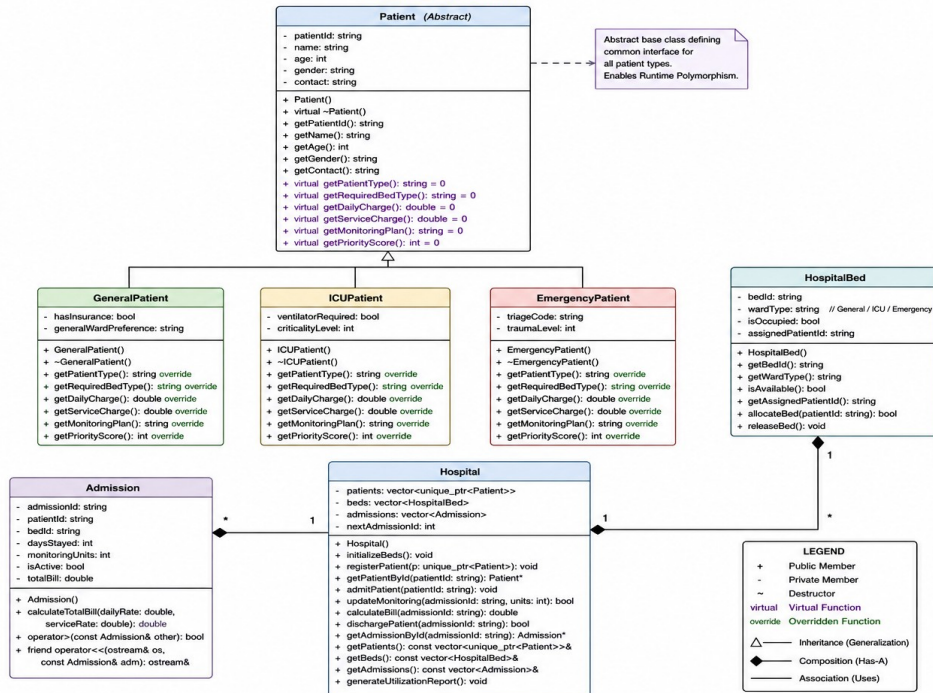


### E.1 Use Case Diagram

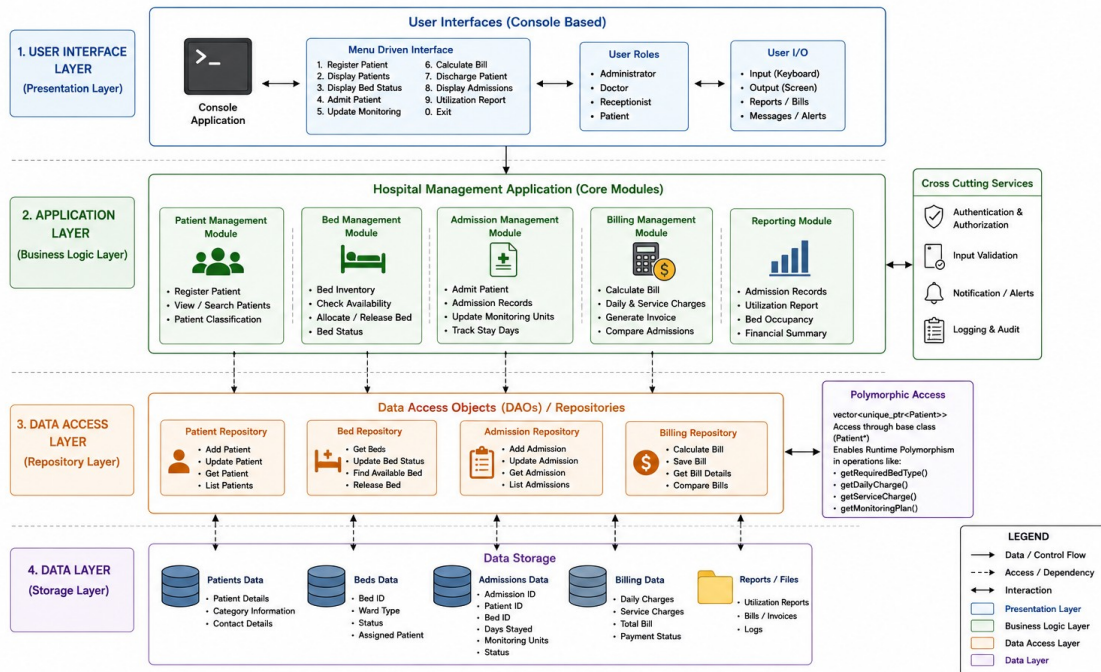
#### SMART HOSPITAL BED MANAGEMENT SYSTEM CLASS DIAGRAM



## SMART HOSPITAL BED MANAGEMENT SYSTEM CLASS DIAGRAM

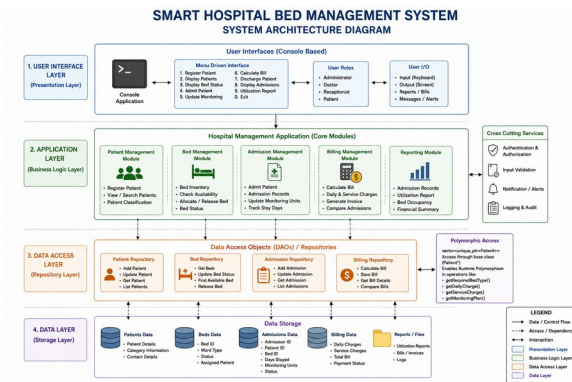


## E.2 Class Diagram SMART HOSPITAL BED MANAGEMENT SYSTEM SYSTEM ARCHITECTURE DIAGRAM

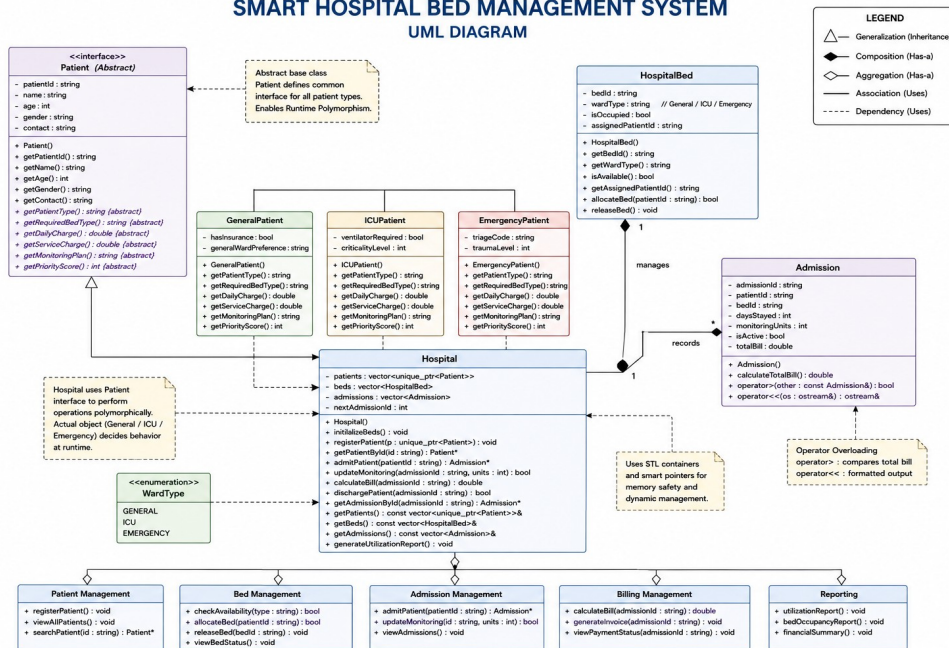


## E.3 System Architecture

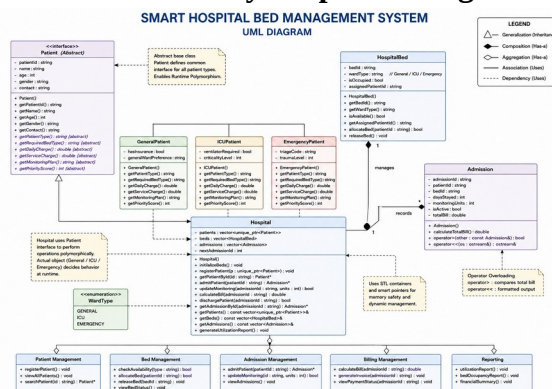




## SMART HOSPITAL BED MANAGEMENT SYSTEM UML DIAGRAM



## E.4 Runtime Polymorphism Diagram



## E.5 Overall Flowchart

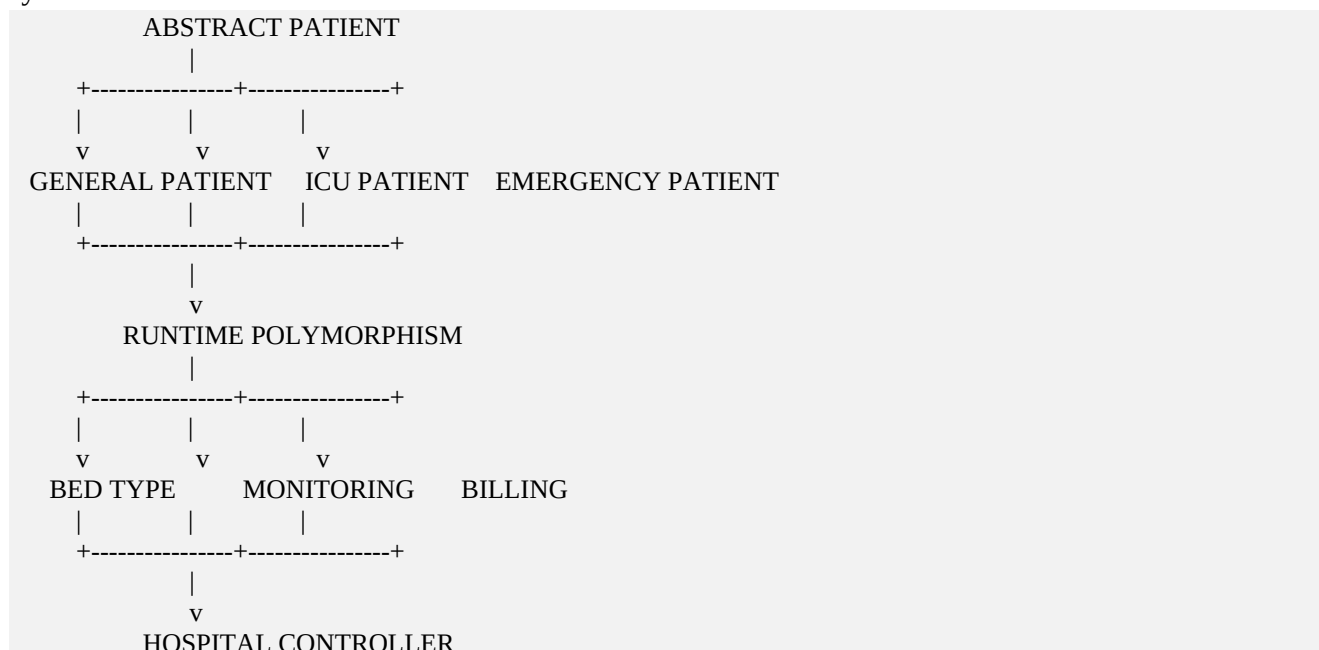
## 42. Final Requirement Audit

**Table 42.1: Final Requirement Verification Audit**

| Requirement            | Implementation           | Evidence               | Status              |
|------------------------|--------------------------|------------------------|---------------------|
| C++ Project            | C++17 source code        | Project files          | FULLY MET           |
| Object-Oriented Design | Multiple classes         | Class diagram          | FULLY MET           |
| Abstraction            | Abstract Patient         | Patient.h              | FULLY MET           |
| Inheritance            | Three derived classes    | UML diagram            | FULLY MET           |
| Runtime Polymorphism   | Virtual functions        | Hospital.cpp           | FULLY MET           |
| Dynamic Binding        | Base-class pointer calls | Execution evidence     | FULLY MET           |
| General Patient        | GeneralPatient           | Registration test      | FULLY MET           |
| ICU Patient            | ICUPatient               | Registration test      | FULLY MET           |
| Emergency Patient      | EmergencyPatient         | Registration test      | FULLY MET           |
| Dynamic Bed Allocation | getRequiredBedType()     | Allocation screenshots | FULLY MET           |
| Monitoring             | Monitoring functions     | Monitoring screenshot  | FULLY MET           |
| Dynamic Billing        | Rate virtual functions   | Billing screenshot     | FULLY MET           |
| Admission Management   | Admission class          | Admission output       | FULLY MET           |
| Bed Release            | releaseBed()             | Discharge evidence     | FULLY MET           |
| Operator Overloading   | > and <<                 | Admission module       | FULLY MET           |
| Smart Pointers         | unique_ptr               | Source code            | FULLY MET           |
| STL Vector             | vector collections       | Source code            | FULLY MET           |
| Input Validation       | Validation routines      | Error screenshot       | FULLY MET           |
| Multi-file Structure   | .h and .cpp files        | Directory screenshot   | FULLY MET           |
| Testing                | Test matrix              | Chapter 27             | FULLY MET           |
| Compilation Evidence   | G++ build                | Figure 26.1            | PENDING SCREENSHOT  |
| Execution Evidence     | Terminal screenshots     | Chapter 28             | PENDING SCREENSHOTS |
| GitHub Repository      | Repository link          | Appendix D             | PENDING LINK        |

### Final Project Summary

The Smart Hospital Bed Management System is a C++17 console application developed to demonstrate how Runtime Polymorphism can be applied to a practical hospital resource-management problem. The system is centered around:



The central idea is that the Hospital controller communicates with patient objects through a common interface while the runtime object determines the actual behavior. This design demonstrates the practical value of Runtime Polymorphism in creating software that is modular, extensible, maintainable, and easier to adapt to future requirements.